Space complexity: L, PSPACE, and NL

Exercise sheet 11 (hints)

Logspace.

Exercise 11.1. Let A, B, and C be languages. Prove the following statements about logspace reductions:

(a) If $A \leq_L B$ and $B \leq_L C$, then $A \leq_L C$.

(b) If $A \leq_L B$ and $B \in \mathrm{L}$, then $A \in \mathrm{L}$.

(c) Suppose that B is P-complete under logspace reductions and $B \in \mathrm{L}$. Prove that $\mathrm{L} = \mathrm{P}$.

Hint: The output of a logspace reduction need not be stored. If the next machine asks for a symbol of that output, rerun the first transducer and count until that position. For (c), apply part (b) to every language in P.

Exercise 11.2. Prove that L is closed under union and intersection: if $A, B \in \mathrm{L}$, then $A \cup B \in \mathrm{L}$ and $A \cap B \in \mathrm{L}$.

Hint: Run the two deciders one after the other. Their work space can be reused.

Exercise 11.3. A word over $\{(,)\}$ is a balanced-parentheses word if every prefix contains at least as many left parentheses as right parentheses, and the whole word contains equal numbers of each. Prove that the language of balanced-parentheses words is in L.

Hint: Scan from left to right, keeping the current difference between the two counts.

Polynomial space.

Exercise 11.4. A fully quantified Boolean formula is true if its quantified variables can be assigned values according to the quantifiers so that the formula evaluates to true. Give an algorithm that decides this problem using polynomial space. Explain why it may take much more than polynomial time.

Hint: Evaluate the quantifiers recursively. An existential quantifier asks whether one of two recursive calls accepts; a universal quantifier asks whether both do. Reuse the same workspace when a recursive call returns.

Exercise 11.5. * Let TQBF-CNF be the problem of deciding the truth of a fully quantified Boolean formula whose unquantified matrix is in conjunctive normal form. Prove that TQBF-CNF is PSPACE-complete under polynomial-time reductions.

Hint: Start with a quantified formula whose matrix is an arbitrary Boolean formula. Introduce one fresh variable for each gate, use clauses to describe what each gate computes, and existentially quantify the new variables after the original prefix.

Exercise 11.6. * Define IN-SPACE-ACCEPTANCE as follows:

Input: A deterministic Turing machine M and a word x.

Question: Does M accept x without ever using a tape cell beyond the first $|x|$ cells?

Prove that IN-SPACE-ACCEPTANCE is PSPACE-complete.

Hint: For membership, count how many configurations fit in the allowed space. For hardness, pad the input so the machine has enough permitted tape cells to run a polynomial-space decider.

Exercise 11.7. * Use the space hierarchy theorem and padding to prove that P $\neq$ SPACE(n), where SPACE(n) is deterministic space bounded by a linear function of the input length.

Hint: Choose a language in SPACE(n^2) *but not in* SPACE(n). *Pad each input to quadratic length; the padded language fits in linear space. If it is in* P*, what does unpadding imply about the original language?*

Nondeterminism. In a nondeterministic computation, a machine can have several possible next moves. Its computation can be viewed as a tree of branches. The machine accepts if at least one branch accepts; a space bound applies to each branch separately.

Exercise 11.8. * Let NSPACE (s(n)) denote the class of languages decided by nondeterministic machines using $O(s(n))$ working space. Use the deterministic space hierarchy theorem and Savitch's theorem to prove that there is a language in SPACE(n^3) that is not in NSPACE (n). You may quote both theorems.

Hint: Savitch's theorem gives NSPACE(n) $\subseteq$ SPACE(n^2). *Choose a language just beyond this deterministic space bound.*

Nondeterministic logspace. The class NL consists of languages accepted by nondeterministic machines using $O(\log n)$ working space.

Exercise 11.9. Prove that NL $\subseteq$ P. Use the graph whose vertices are configurations of a nondeterministic logspace machine and whose edges are its possible moves.

Hint: There are polynomially many configurations. A deterministic graph search can test whether an accepting configuration is reachable from the start configuration.

Exercise 11.10. Prove that NL is closed under union and intersection.

Hint: For union, choose which machine to simulate. For intersection, simulate the two machines in sequence, reusing the workspace.

Exercise 11.11. Prove that DIRECTED-REACHABILITY is in NL. Here the input is a directed graph G and two vertices s and t, and the question is whether there is a directed path from s to t. The lecture also states that this problem is NL-complete under logspace reductions. Explain why a deterministic logspace algorithm for it would imply L = NL, and whether we know that this equality holds.

Hint: Guess a path one edge at a time and keep a counter. If a path exists, one with no repeated vertices exists. For the last part, use completeness and Exercise 11.1(b).

Exercise 11.12. * Prove that NON-BIPARTITE, the problem of deciding whether an undirected graph is not bipartite, is in NL. Does this also prove that BIPARTITE is in NL? Briefly explain.

Hint: An undirected graph is not bipartite exactly when it has an odd cycle. What short information is needed to check a guessed walk around such a cycle?

From Exercise sheets 9 & 10.

Exercise 11.13. * A nondeterministic Turing machine may have several possible transitions for a state and observed symbol. It accepts an input if some computation halts in the accepting state, and rejects if all computations reject. It runs in polynomial time if every computation halts within a polynomial number of steps.

Prove that $L \in$ NP if and only if a polynomial-time nondeterministic Turing machine decides L.

Hint: In one direction, let the machine guess a certificate and then run the verifier. How do you bound every branch? In the other direction, what information lets a deterministic verifier replay one chosen computation? The machine has a fixed finite number of transition choices.

Exercise 11.14. * Let NEXP be the class of languages decided by exponential-time nondeterministic Turing machines. Prove that if P = NP, then EXP = NEXP.

Hint: Consider a computation together with a time bound written in unary. Checking a proposed accepting computation is then polynomial in this longer input. Under the assumption, what deterministic algorithm becomes available? Apply it to an exponentially large time bound, carefully distinguishing the original input length from the padded length.